

Modelli linguistici e Q-learning: inizializzare la tabella dei valori nell'ambiente MiniGrid-DoorKey

Pietro Fazion
Università di Verona

29 settembre 2026

Sommario

L'ambiente MiniGrid-DoorKey assegna la ricompensa solo quando l'agente raggiunge l'obiettivo finale. Il Q-learning classico impiega molti episodi per registrare il primo successo. Questo documento valuta una singola ipotesi. L'ipotesi prevede l'uso di un modello linguistico per stimare il valore ottimo su un gruppo ridotto di stati. Il sistema inserisce questi valori nella tabella Q prima di iniziare l'addestramento.

L'esperimento sul seed 1337 utilizza 20 coppie stato-azione relative ai passaggi critici. Questa configurazione raggiunge un tasso di successo del 100% con 17 passi medi. La configurazione di controllo senza stime iniziali registra un tasso dello 0%. Il modello linguistico da 120 miliardi di parametri presenta un errore medio di 0.0122. Il modello indica l'azione corretta nel 94.1% dei casi. Il trasferimento dell'apprendimento su mappe nuove non produce risultati. Il testo cita i file di log per la verifica dei dati.

Indice

1	Il problema	3
2	Struttura della repository	3
3	L'ambiente e il valore ottimo	3
4	Selezione degli stati	4
4.1	Tipologie di nodi critici	4
4.2	Campionamento della progressione	5
5	Interrogazione del modello	5
6	Precisione dei calcoli	6

7	Qualità dell'azione suggerita	10
8	Impatto delle stime iniziali	10
9	Scelta dei parametri	14
10	Limiti della ricerca	15
11	Sviluppi futuri	15
A	Appendice: ambiente FrozenLake	16

1 Il problema

L'algoritmo Q-learning propaga il valore dall'obiettivo agli stati precedenti [1]. Le definizioni standard di Processo Decisionale di Markov e l'equazione di Bellman restano invariate.

La ricompensa arriva raramente nelle fasi iniziali. La tabella mantiene valori nulli per molto tempo. L'agente non riceve indicazioni per prendere decisioni.

L'esperimento stima il valore V_{opt} con un modello linguistico su pochi stati. Il codice inserisce i valori nella tabella Q iniziale. L'addestramento prosegue con il Q-learning classico. Il processo non usa ricompense aggiuntive. L'obiettivo temporale resta invariato.

I valori iniziali guidano l'agente nei punti decisivi. Questo compensa la mancanza di ricompense immediate.

L'esperimento principale valuta il seed 1337. L'addestramento dura 2000 episodi. Il tasso di apprendimento α vale 0.07. Il fattore di sconto γ vale 0.9. La configurazione usa 20 coppie stato-azione generate dal modello. Questa configurazione completa la mappa nel 100% dei casi con 17 passi di media. La configurazione senza stime iniziali fallisce in ogni tentativo e consuma 450 passi. I dati risiedono nella cartella `src/output/agents/`. I file sono `qtable_llmininit_seed_1337_gpt-oss_120b_llmininit.json` e `qtable_llmininit_seed_1337_gpt-oss_120b_vsame.json`. La Sezione 8 discute i risultati.

2 Struttura della repository

Il codice sorgente e i dati risiedono nella repository ufficiale del progetto. L'indirizzo web è <https://github.com/ttetrao/llm-guided-rl/tree/main>. La cartella `src/` contiene i moduli operativi. I programmi si avviano da questa posizione. Il comando è `python3-mgraph.nome_modulo`. Lo schema si applica a tutti i pacchetti. Il pacchetto `src/graph/` costruisce il grafo del processo decisionale e seleziona gli stati. Il pacchetto `src/llm/` interroga i modelli linguistici. Il pacchetto `src/evaluate/` genera le metriche e i grafici. Il pacchetto `src/agent/` esegue il Q-learning e gestisce i controlli. Il pacchetto `src/env/` contiene gli adattatori per l'ambiente e definisce le fasi. Il pacchetto `src/docs/` raccoglie le istruzioni testuali per i modelli.

Il modulo `src/paths.py` gestisce le destinazioni di salvataggio. I dati finiscono sotto `src/output/`. La cartella `cache` conserva i grafi. La cartella `llm` ospita le risposte dei modelli. La cartella `agents` raccoglie i dati degli addestramenti. Le cartelle `evaluate` contengono i grafici di riepilogo. Il file `src/agent/doorkey_qtable_llmininit2.py` produce i confronti diretti. I documenti di tesi risiedono in `thesis/`.

3 L'ambiente e il valore ottimo

Il progetto utilizza l'ambiente `MiniGrid-DoorKey-8x8-v0` [2]. Le regole sono originali. L'agente raccoglie la chiave. L'agente apre la porta con l'azione `toggle`. L'agente

raggiunge l'obiettivo in coordinate (6,6). La ricompensa vale 1 solo sul bersaglio. La ricompensa vale 0 in ogni altra cella. Il limite massimo di passi è 450.

Lo stato interno del grafo richiede cinque variabili. Le variabili sono la posizione, la direzione dell'agente, il possesso della chiave e lo stato della porta. La posizione dei muri dipende dal seed. Queste cinque variabili descrivono la mappa in modo univoco per ogni seed.

Lo script `src/graph/mdp_graph.py` genera il grafo. Il calcolo considera sette azioni per ogni stato. Il processo determina la cella di arrivo e la ricompensa. Il grafo contiene 480 nodi per ogni seed. Il bersaglio si trova sempre in (6,6). Il calcolo utilizza il metodo della value iteration e un fattore di sconto 0.99 per determinare il valore ottimo V_{opt} . Questo calcolo fornisce il parametro di valutazione per il modello linguistico. Il processo esclude gli stati finali dalle interrogazioni. Il sistema ignora l'azione nulla.

Lo script `src/env/view_wrapper.py` traccia il progresso dell'agente. Lo stato `find_key` indica la ricerca della chiave. Lo stato `open_door` indica il possesso della chiave e la porta chiusa. Lo stato `reach_goal` indica il via libera verso l'obiettivo. L'agente deve rispettare questa sequenza. Due posizioni uguali con stati di avanzamento diversi richiedono azioni diverse.

4 Selezione degli stati

L'interrogazione completa dei 480 nodi richiede molto tempo. Molte celle di passaggio condividono lo stesso valore. Un parametro definisce il numero massimo di interrogazioni. Il file `src/graph/doorkey_states.py` seleziona gli stati utili.

4.1 Tipologie di nodi critici

La funzione `extract_bottleneck` isola i nodi critici dal grafo. Il processo non richiede l'esecuzione di partite. I nodi si dividono in sei categorie.

1. `on_key`: la cella contiene la chiave. Questa categoria produce 8 nodi.
2. `on_door`: la cella contiene la porta aperta. Questa categoria produce 8 nodi.
3. `adj_door`: l'agente fronteggia la porta chiusa da una cella adiacente. Questa condizione permette di interagire. La categoria produce da 3 a 4 nodi.
4. `door_toggle`: l'azione `toggle` apre la porta con successo. La categoria conta sempre 2 nodi.
5. `key_take`: l'azione di raccolta acquisisce la chiave. La categoria conta da 2 a 8 nodi in base alla posizione dei muri.
6. `goal_entry`: un passo separa l'agente dall'obiettivo. La categoria fornisce da 4 a 8 nodi.

Queste categorie mappano posizioni, condizioni di azione e stati finali. I nodi totali variano tra 30 e 34. Questo valore rappresenta meno del 10% dell'intero grafo.

4.2 Campionamento della progressione

I nodi critici marcano i passaggi obbligati. I nodi di progressione fotografano l'evoluzione dell'agente nel tempo. L'agente visita molte celle inutili all'inizio. L'agente si concentra sul percorso ottimale alla fine. La funzione `run_pilot` addestra un agente senza aiuti esterni. I parametri sono $\alpha = 0.3$ e $\gamma = 0.99$. L'esplorazione decade lentamente.

Il programma salva gli stati visitati quando il successo medio tocca soglie specifiche. Le soglie sono 0.3, 0.5, 0.7, 0.8 e 1.0. I salvataggi per i valori bassi includono tutti i percorsi tentati. I salvataggi per i valori alti aggiungono deviazioni casuali. Questo accorgimento impedisce di registrare solo la sequenza perfetta. Il seed 1337 attiva le soglie agli episodi 93, 115, 137, 150 e 175. Ogni attivazione salva circa 300 nodi.

I seed 47952 e 74839 completano il gioco al primo tentativo per puro caso. Tutte le soglie si attivano simultaneamente. Le liste contengono pochissimi nodi. Il codice riporta questi eventi per tracciamento.

La funzione `allocate` riserva il primo spazio per i nodi critici. Il programma divide il resto della capienza tra i nodi di progressione usando il metodo dei resti maggiori.

5 Interrogazione del modello

Il programma calcola il valore per l'azione Q a partire dal valore dello stato V . Il testo di istruzione risiede in `src/docs/doorkey/it/prompt.txt`. I file `src/llm/query_gpt.py` e `src/llm/query_gemma.py` gestiscono le chiamate.

Il modello riceve le regole del gioco e la legenda della mappa. Il testo include le definizioni per il calcolo del valore. Il programma invia la mappa corrente e le sei mappe relative alle possibili mosse. Il modello linguistico valuta solo la distanza dall'obiettivo per ogni opzione. L'espressione usata è γ^d con $\gamma = 0.99$. Il calcolo sfrutta il determinismo del gioco. Il valore esatto di uno stato distante d mosse corrisponde a γ^d . Il modello non può adottare stime casuali. Il modello risponde nel formato JSON. Ogni risposta include l'applicazione della regola, il conteggio dei passi e il calcolo numerico. Il programma scarta le risposte anomale o errate.

Il test principale impiega un modello da 120 miliardi di parametri sul seed 1337. I file occupano la cartella `src/output/llm/`. Il file `llm_results_doorkey_states_8x8_seed1337_gpt-oss_120b.json` contiene 500 righe di calcolo. Le risposte valide sono 85 su 99 richieste. Il codice attende 65 secondi tra una chiamata e l'altra. Il sistema ritenta 3 volte in caso di errore.

Un secondo test analizza cinque seed differenti. Il file `llm_results_doorkey_states_8x8_seeds13507-19920-32455-54937-69693_gpt-oss_120b.json` raccoglie 111 stati validi e 629 righe.

6 Precisione dei calcoli

Lo script `src/evaluate/evaluate_llm.py` calcola le metriche. I risultati riempiono il file `src/output/evaluate/log_valutazione.txt`. L'analisi confronta il valore calcolato dal modello linguistico con il valore reale della mappa.

Risultati sul seed 1337:

- L'errore medio ammonta a 0.0122. Il bias segna -0.0104 . Il 97.4% delle risposte ha un errore inferiore a 0.05.
- Gli indici di correlazione superano lo 0.9.

I nodi critici generano l'errore più alto con 0.0140. La ricerca della chiave risulta la fase più soggetta a imprecisioni.

I test sui cinque seed restituiscono un errore medio di 0.0153 con un bias di -0.0116 . Il 96.8% si posiziona sotto la soglia di 0.05.

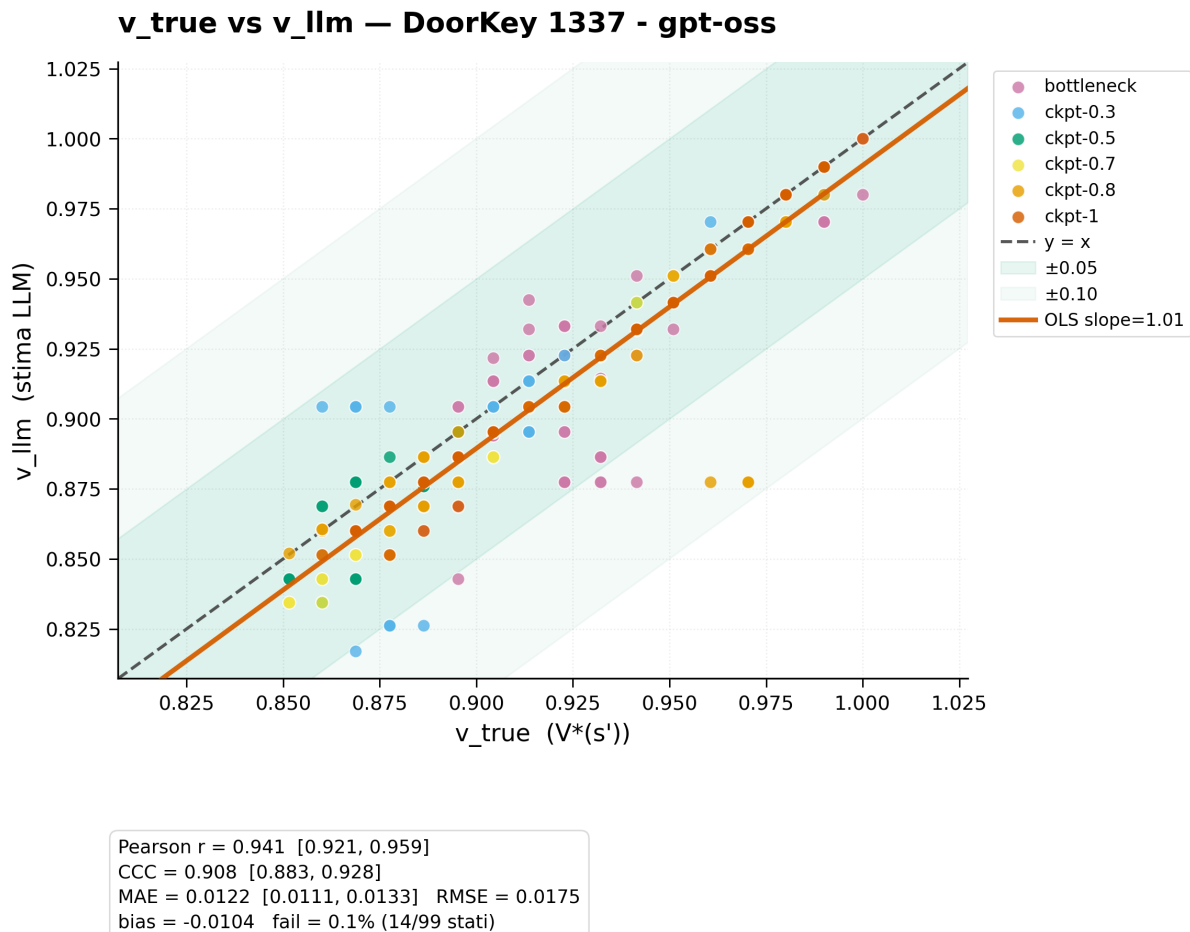


Figura 1: Il grafico mostra la correlazione tra i valori calcolati dall'algoritmo classico e le stime del modello linguistico. L'asse orizzontale rappresenta i valori reali. L'asse verticale rappresenta le stime. Una stima perfetta posiziona i punti sulla linea diagonale. I punti si concentrano lungo questa linea. Questo indica una buona precisione del modello.

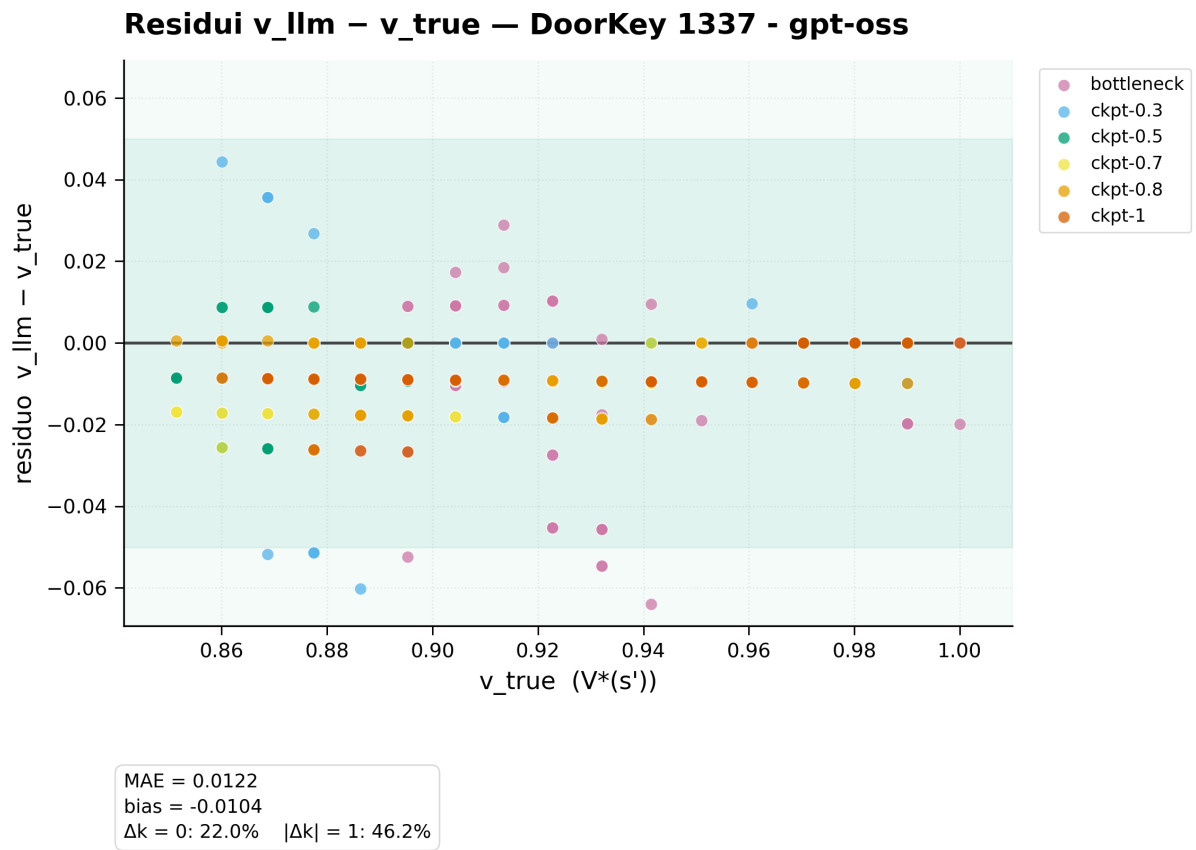


Figura 2: Il grafico analizza gli errori del modello. L'asse orizzontale mostra i valori attesi. L'asse verticale mostra la differenza tra il valore stimato e il valore reale. I punti sotto lo zero indicano una sottostima. Il modello tende a sottostimare i valori.

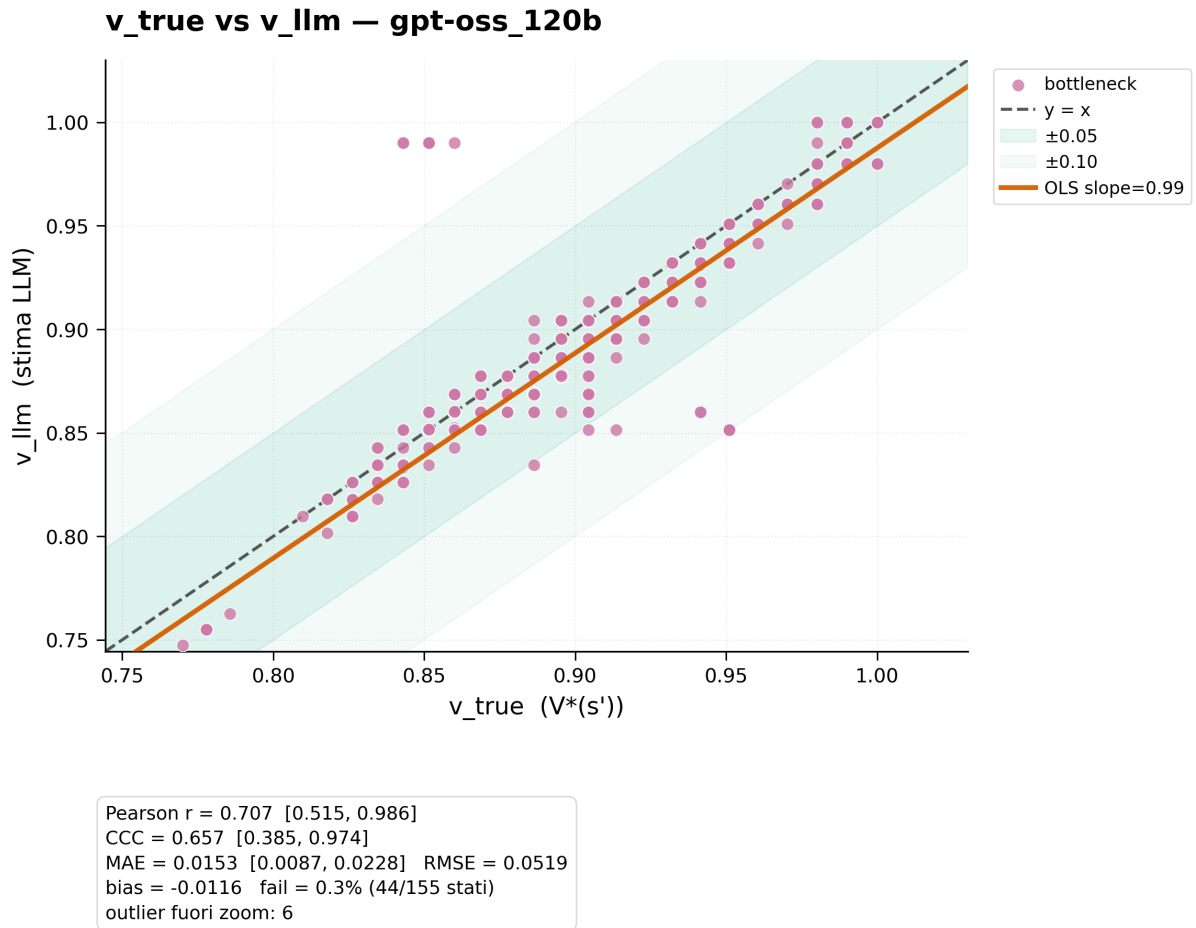


Figura 3: Il grafico estende l'analisi a cinque scenari differenti considerando solo gli stati critici. La struttura ricalca il grafico precedente sui valori stimati e reali. I punti risultano maggiormente distanti dalla linea diagonale. Questo indica una minore precisione del modello nel valutare scenari diversi.

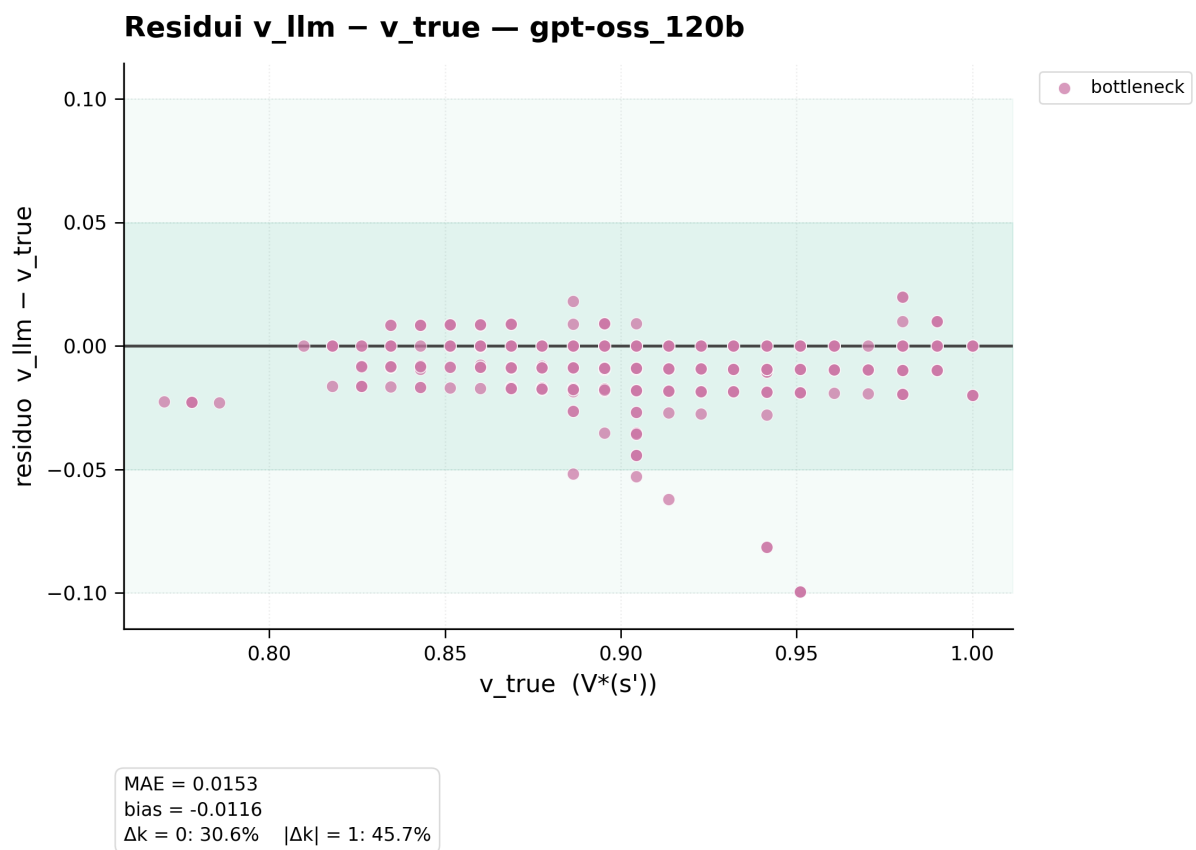


Figura 4: Il grafico riporta gli errori del modello sui cinque scenari. L'asse orizzontale mostra i valori attesi. L'asse verticale mostra la differenza tra stima e realtà. La distribuzione dei punti appare più ampia rispetto al caso singolo. La tendenza a sottostimare si conferma.

Il bias negativo di -0.01 corrisponde all'errore sistematico di circa un passo. Le Figure 1 e 3 mostrano la dispersione dei punti.

7 Qualità dell'azione suggerita

L'algoritmo valuta la correttezza dell'azione proposta dal modello. La valutazione tollera piccole imprecisioni numeriche causate dal limite dei decimali. Dati per il seed 1337:

- L'azione suggerita risulta essere la migliore nel 94.1% dei casi. L'azione casuale ottiene il 16.7%.
- La perdita attesa dovuta a scelte sub-ottimali equivale a 0.0011.
- La correttezza sui nodi critici raggiunge l'82.8%.

Il modello sbaglia i conteggi assoluti dei passi ma identifica l'azione preferibile. Le Figure 5 e 6 riportano il dettaglio per categorie.

L'analisi sui cinque seed evidenzia l'azione migliore nel 91.9% delle situazioni.

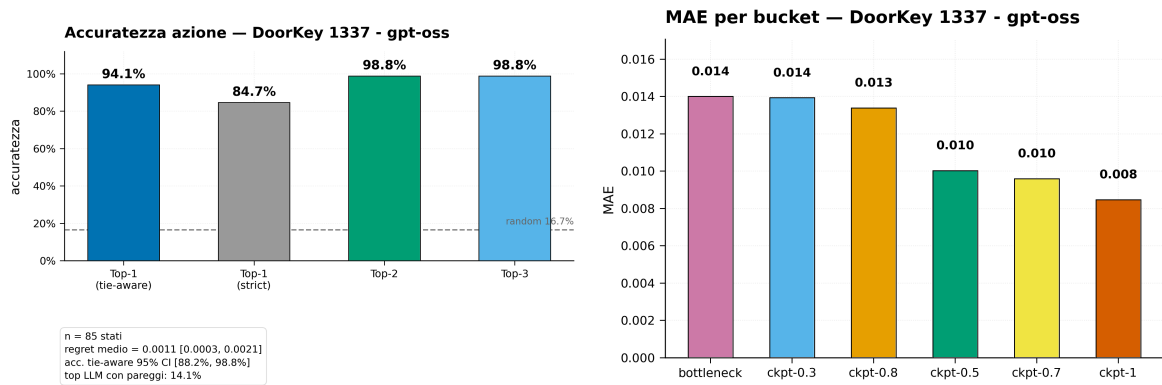


Figura 5: La figura contiene due grafici a barre. Il primo grafico mostra la percentuale di risposte corrette del modello per diverse categorie di stati. Il secondo grafico mostra l'entità dell'errore per le stesse categorie. Gli stati definiti critici registrano gli errori più alti.

8 Impatto delle stime iniziali

Il codice sorgente per l'agente è `src/agent/doorkey_qtable_llminit.py`. L'agente ragiona su un sistema di coordinate relative all'obiettivo intermedio. Questo approccio unifica il comportamento su seed diversi. Il sistema inserisce per prime le stime calcolate. Le celle rimanenti partono da zero. Il Q-learning procede elaborando le celle in ordine decrescente di valore stimato.

La funzione `optimal_policy_metrics` valuta il comportamento dell'agente rispetto alla soluzione teorica perfetta.

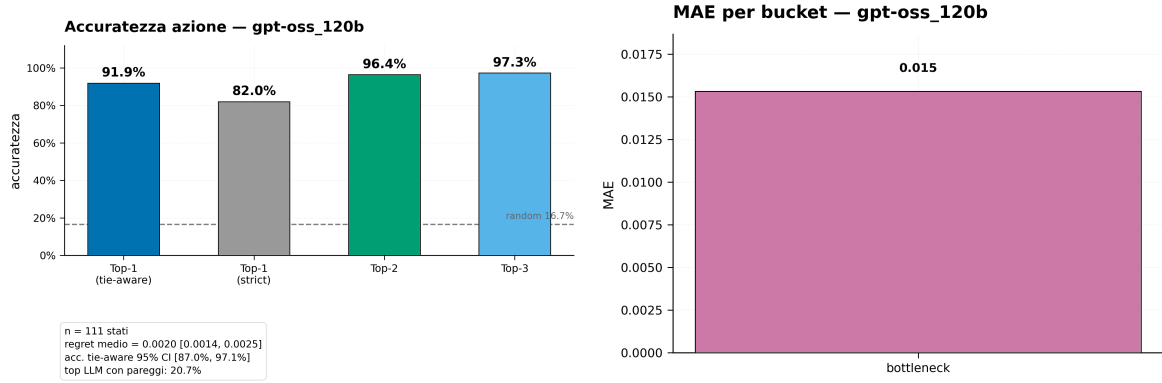


Figura 6: La figura presenta due grafici a barre relativi a cinque scenari. Il primo grafico mostra quante volte il modello suggerisce l'azione migliore o la seconda migliore. Il secondo grafico mostra l'errore medio diviso per categorie.

La prima metrica definisce l'accordo con la politica ottima. Questo parametro conta le volte in cui la decisione dell'agente coincide con l'azione ideale calcolata a priori. Un valore pari a 1 indica una totale aderenza alla strategia perfetta. Un valore basso indica scelte sub-ottimali. Questo numero valuta la precisione delle regole apprese.

La seconda metrica calcola la perdita di valore media. Questo indicatore quantifica il costo degli errori. Ogni mossa sbagliata allontana l'agente dall'obiettivo e abbassa il punteggio atteso. Una perdita media di 0.0067 si traduce nella perdita di una frazione di turno di gioco. Valori prossimi allo zero segnalano traiettorie eccellenti. Valori alti evidenziano percorsi tortuosi.

Il sistema elabora entrambe le metriche esclusivamente sui nodi esplorati durante le partite effettive.

I test utilizzano il seed 1337 per 2000 episodi. Il parametro α vale 0.07. Il parametro γ vale 0.9. Il decadimento vale 0.99. Il codice inserisce i 20 nodi critici principali.

- L'agente inizializzato completa la mappa in 17 passi con un accordo di 0.585.
- L'agente standard con gli stessi parametri non completa la mappa e impiega 450 passi.
- L'agente standard con parametri intensivi ($\alpha = 0.25$, $\gamma = 0.99$) completa la mappa in 16 passi.

Il controllo senza inizializzazione fallisce con gli stessi parametri base. Le stime innescano la convergenza anticipata dell'algoritmo (Figure 7 e 8).

Il secondo ciclo di test esegue l'addestramento su cinque seed differenti. Le mappe sono 13507, 19920, 32455, 54937 e 69693. L'addestramento impiega le coordinate relative e prosegue per 4000 turni. I parametri salgono a $\alpha = 0.1$ e $\gamma = 0.8$. Il sistema inserisce 500 stime iniziali. Il codice verifica le prestazioni sul seed conosciuto 13507 e sul nuovo seed 6499. Il tasso di successo sulle mappe apprese raggiunge il 100% con 23 passi di media. Il tasso di successo sulla mappa nuova si ferma allo 0%. I controlli senza stime registrano risultati nulli su tutta la linea. Cinque layout non consentono al sistema di imparare regole generali esportabili su nuovi livelli (Figure 9 e 10).

DoorKey 8x8 seed 1337 | 2000 episodi | llm_results_doorkey_states_8x8_seed1337_gpt-oss_120b.json | LLM-init vs Vanilla (same hp)

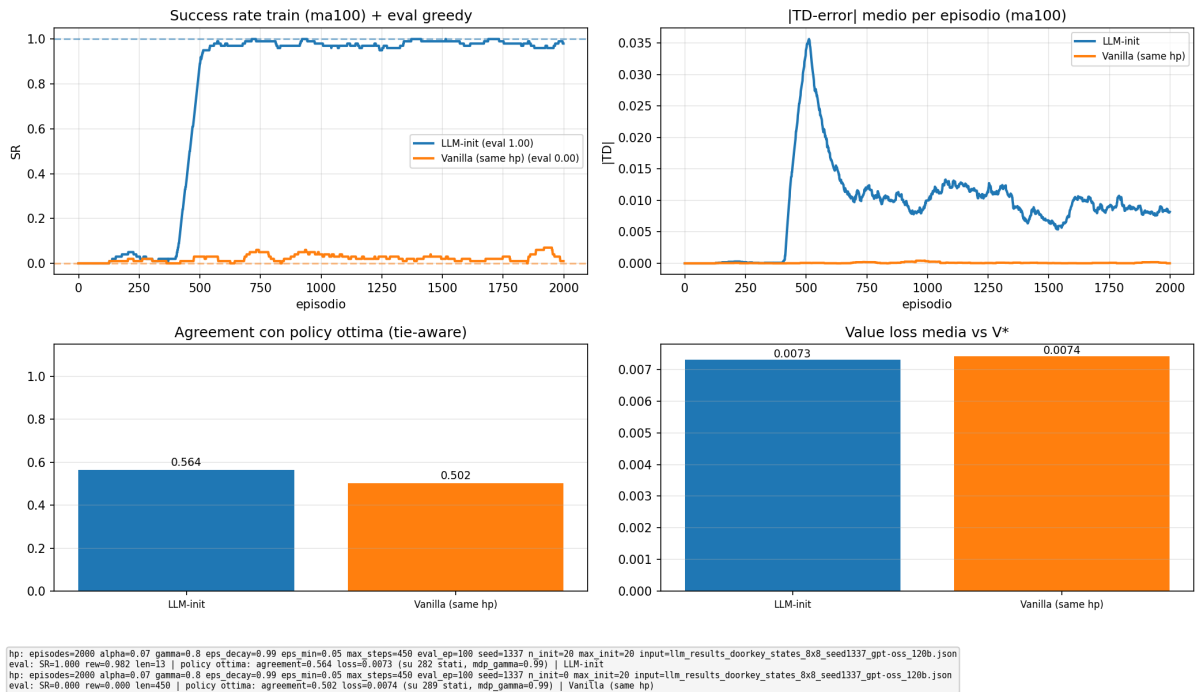


Figura 7: Il grafico confronta le prestazioni di due agenti nel tempo. L'asse orizzontale rappresenta il numero di episodi giocati. L'asse verticale misura il tasso di successo. La linea relativa all'agente con stime iniziali sale rapidamente a uno. La linea dell'agente senza stime iniziali rimane piatta a zero. Questo dimostra l'utilità delle stime iniziali.

DoorKey 8x8 seed 1337 | 2000 episodi | llm_results_doorkey_states_8x8_seed1337_gpt-oss_120b.json | LLM-init vs Vanilla-std

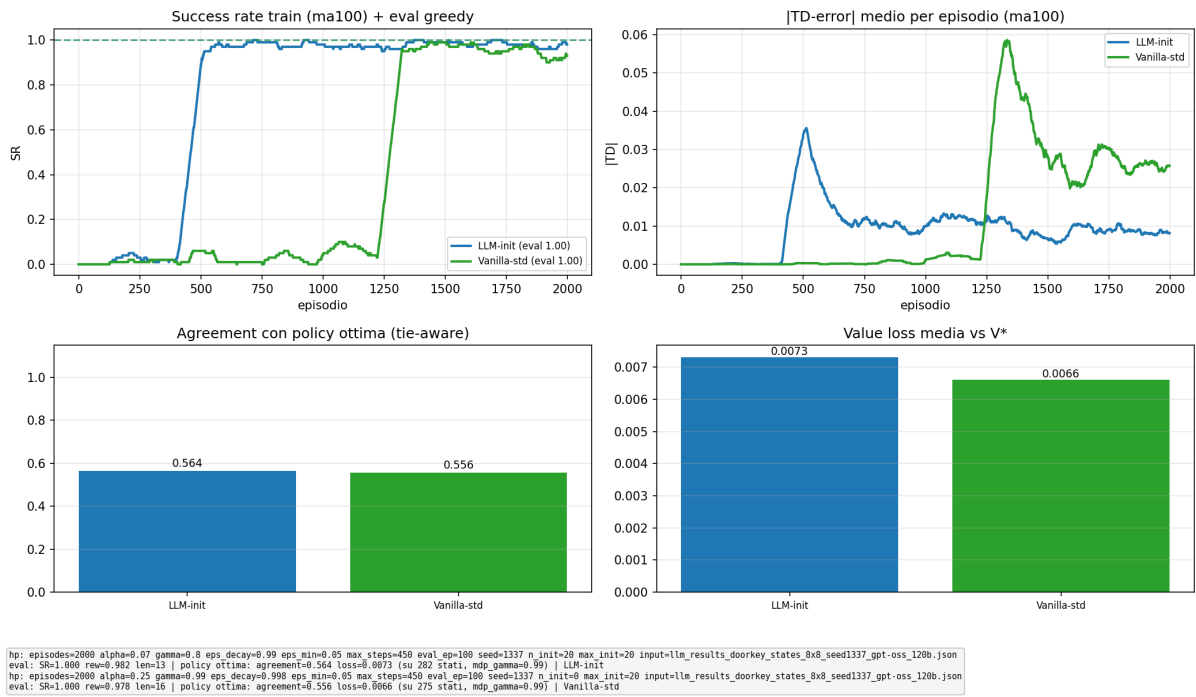


Figura 8: Il grafico confronta l'agente potenziato con un agente standard ottimizzato. Entrambe le linee raggiungono il tasso di successo massimo. L'asse orizzontale conta gli episodi. L'asse verticale mostra il raggiungimento dell'obiettivo. Le prestazioni risultano simili.

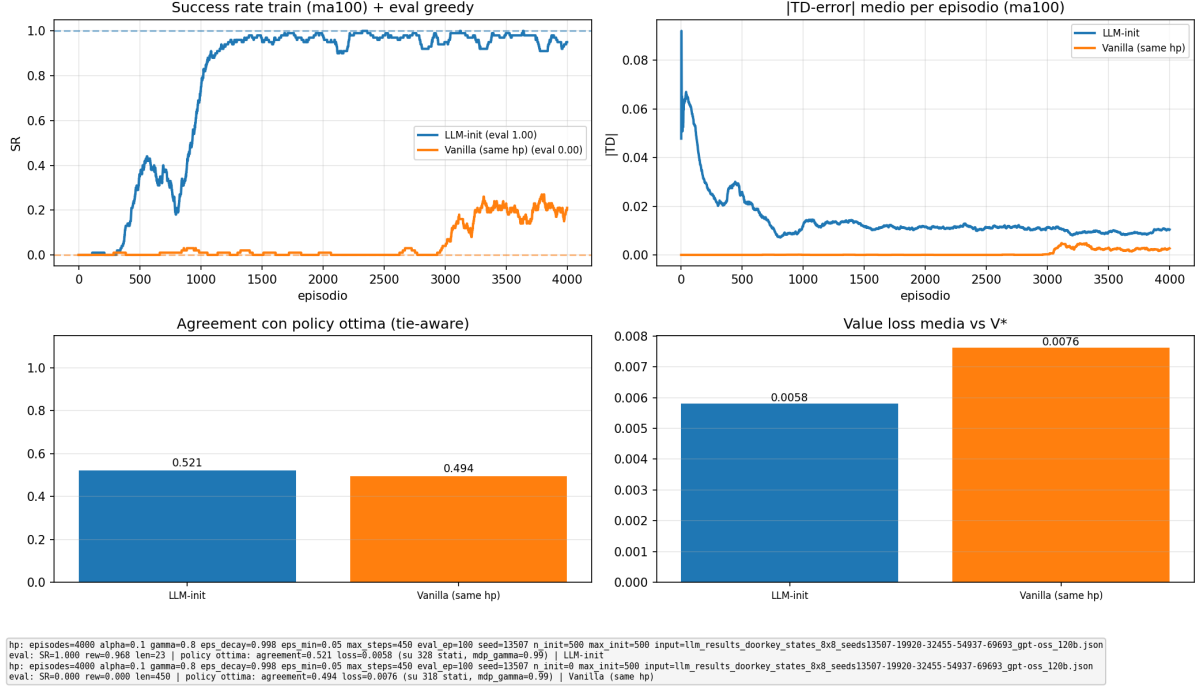


Figura 9: Il grafico mostra le prestazioni su mappe mai viste durante l'addestramento. L'asse orizzontale conta gli episodi. L'asse verticale indica il successo. Nessun agente riesce a risolvere le nuove mappe. L'apprendimento non si trasferisce a scenari sconosciuti.

I test con quantità variabili di stati mostrano come un numero maggiore di nodi accelera l'apprendimento. Il numero finale di passi converge sempre verso il minimo indispensabile.

9 Scelta dei parametri

La configurazione base impiega $\alpha = 0.07$, $\gamma = 0.9$, limite esplorativo minimo 0.05 e blocco a 450 passi.

- Il parametro γ scende a 0.9. I valori alti appiattiscono la distanza numerica tra due celle vicine. Un valore di 0.9 aumenta lo scarto e facilita la scelta dell'azione migliore.
- Il programma adotta un tasso α ridotto. La tabella parte già popolata di valori sensati. Un tasso di aggiornamento massiccio distruggerebbe queste stime prematuramente. I test con $\alpha = 0.9$ mostrano un avvio rapido ma il sistema si stabilizza con ritardo.
- Il limite sui passi consente i vagabondaggi esplorativi nelle prime fasi senza interrompere la partita prematuramente.

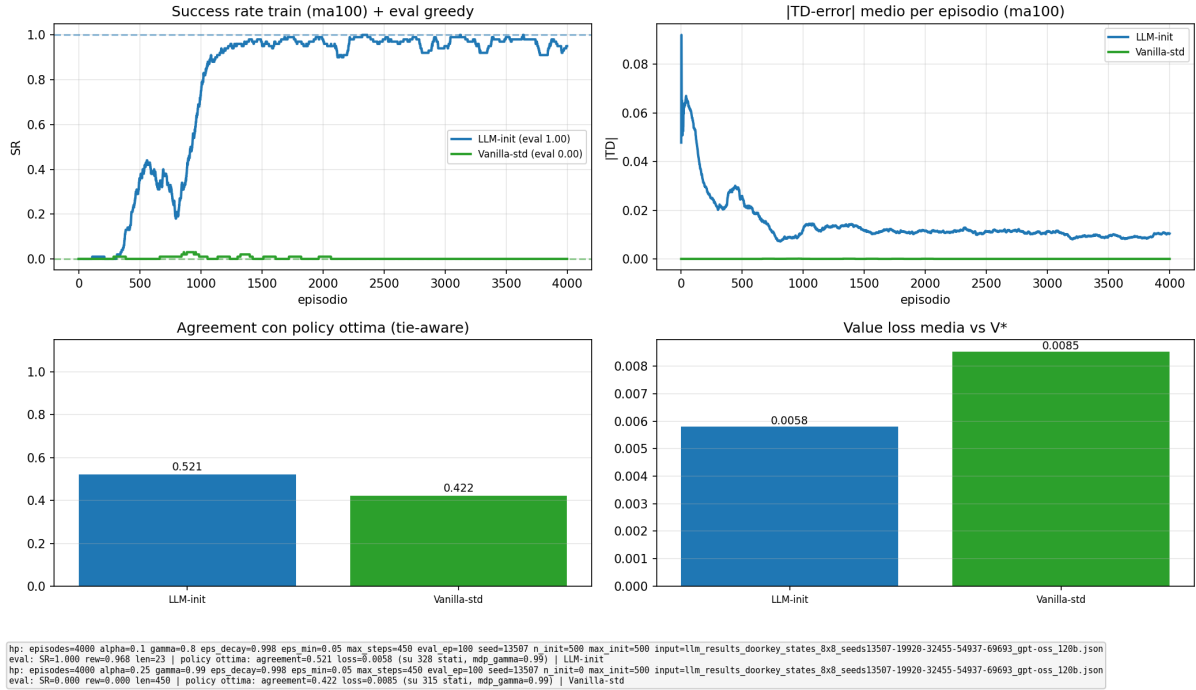


Figura 10: Il grafico riporta un ulteriore test su mappe sconosciute. L'analisi confronta l'agente standard ottimizzato con le nuove mappe. Il tasso di successo rimane nullo per tutti i casi. Nessuna configurazione generalizza l'apprendimento.

10 Limiti della ricerca

Il problema principale consiste nell'incapacità di risolvere mappe mai viste. Il sistema unifica le coordinate. I muri e gli ostacoli variano per ogni schema. Il modello linguistico impiega molto tempo per calcolare le valutazioni per singolo nodo. La copertura parziale dei nodi funziona bene su mappe contenute. L'impatto si riduce drasticamente su mappe vaste. L'esecuzione dell'algoritmo preliminare su due seed produce statistiche anomale dettate dal caso.

11 Sviluppi futuri

I risultati sul singolo seed dimostrano l'efficacia del meccanismo. L'agente apprende più velocemente dei modelli standard. L'esperimento su cinque mappe fallisce nel generare regole universali per mappe nuove. Tre punti necessitano di ulteriore sviluppo.

1. **Estensione del set di addestramento.** L'uso di molte decine di seed permetterebbe all'agente di imparare a gestire gli ostacoli imprevisti.
2. **Modelli dedicati.** Una rete neurale specializzata in stime numeriche ridurrebbe i tempi di attesa. La tabella iniziale si popolerebbe molto più in fretta.

3. **Stima in tempo reale.** Il sistema potrebbe interrogare il modello solo durante l'esplorazione di celle nuove. Questo eviterebbe il calcolo anticipato di tutti gli stati.

I test attuali rappresentano prove di fattibilità eseguite in tiratura singola per ridurre i costi computazionali.

A Appendice: ambiente FrozenLake

L'ambiente **FrozenLake-v1** introduce esiti probabilistici sulle mosse. Il sistema usa un modello linguistico sui 18 nodi critici.

I risultati con 120 miliardi di parametri riportano un errore elevato di 0.1228. Le selezioni corrette si fermano al 55.6%. I modelli faticano a calcolare la media delle probabilità tra esiti incerti tramite ragionamento testuale. Il problema necessita di altre verifiche.

Riferimenti bibliografici

- [1] Sutton, R. S., Barto, A. G. *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- [2] Chevalier-Boisvert, M. et al. MiniGrid and MiniWorld. *NeurIPS* 36, 2023.